

初級編 GPUプログラミングの概要

次世代HPC・AI研究開発支援センター(HAIRDESC)

Section 1.

GPUとは

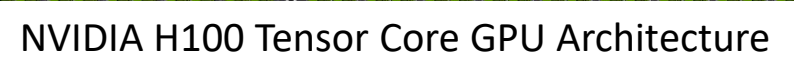
GPU(Graphics Processing Units)

- 元々は画像処理を行うために特化していた専用プロセッサを汎用計算にも使えるように拡張
 - GPGPU: General-Purpose computing on GPU
 - 最近のGPUには(主に深層学習用に)行列積向けユニットも
- 高い演算性能, 太いメモリバンド幅, 消費電力あたり性能も高い
 - 2010年頃のGPUコンピューティング黎明期には安かったが, 最近はとても高価
- 多数のコア(最新GPUは1万越え)を搭載した並列計算機
 - NVIDIA H100 (SXM): 64×132 SMs = 8448 FP64 (16896 FP32) cores
 - NVIDIA B200: 64×148 SMs = 9472 FP64 (18944 FP32) cores
 - AMD MI300A: 64×228 CUs = 14592 Stream Processors
 - AMD MI300X: 64×304 CUs = 19456 Stream Processors
- ざっくり言うと, コア数の10倍以上の並列度があると性能を発揮しやすい
- スレッド並列(e.g., OpenMP)的考えが分かっていると良い



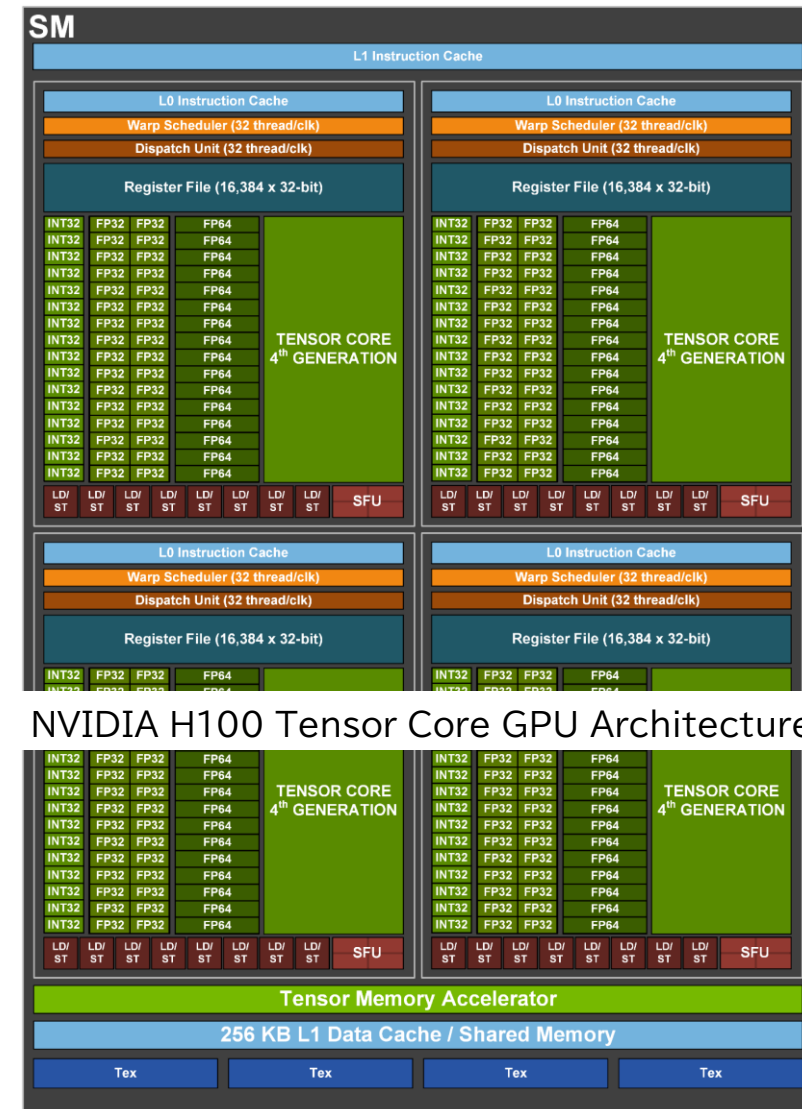
Green 500 Ranking (Nov, 2025)

	TOP 500	System	Accelerator	Cores	HPL Rmax (Pflop/s)	Power (kW)	GFLOPS/W
1	420	KAIROS, CALMIP/CNRS, France	NVIDIA GH200	13,056	3.05	46	73.282
2	171	ROMEO-2025, ROMEO HPC Center - Champagne-Ardenne, France	NVIDIA GH200	47,328	9.86	160	70.912
3	225	Levante GPU extension, DKRZ, Germany	NVIDIA GH200	35,904	6.75	110	69.426
4	213	Isambard-AI phase1, U. Bristol, UK	NVIDIA GH200	34,272	7.42	117	68.835
5	286	Otus (GPU Only), U. Paderborn, Germany	NVIDIA H100 80GB	19,440	4.66		68.177
6	73	Capella, TU Dresden ZIH, Germany	NVIDIA H100 94GB	85,248	24.06	445	68.053
7	334	SSC-24 Energy Module, Samsung, Korea	NVIDIA H100 80GB	11,200	3.82	69	67.251
8	96	Helios GPU, Cyfronet, Poland	NVIDIA GH200	89,760	19.14	317	66.948
9	399	AMD Ouranos, Atos, France	AMD Instinct MI300A	16,632	2.99	48	66.464
10	70	Portage, HPE, United States	AMD Instinct MI300A	129,024	24.50	370	66.277
19	4	JUPITER Booster, Julich, Germany	NVIDIA GH200	4,801,344	1,000.00	15,794	63.316
46	51	TSUBAME 4.0, Science Tokyo, Japan	NVIDIA H100 94GB SXM5	172,800	39.62	816	48.565
50	42	Miyabi-G, JCAHPC, Japan	NVIDIA GH200	221,952	46.80	983	47.588
105	153	PRIMEHPC FX1000, Central Weather Administration, Taiwan	汎用CPU最上位: Fujitsu A64FX	184,320	11.16	674	16.57
115	85	Teton, Idaho National Laboratory, United States	x86 CPU最上位: AMD EPYC 9965	393,216	20.77	1,565	13.27



(NVIDIAの)GPUの構成(2/2)

- SM: Streaming Multiprocessor
 - AMDの場合には Compute Unit (CU)
 - Intelの場合には Execution Unit (EU)
 - 呼び名は違うが, コアの塊が複数という構成は共通
- SMの中での構造は気にしなくても性能が出せる
 - 注: 32スレッドのグループ(ワープ)内で同じ動作をするように意識しておく
 - 32という数字はNVIDIAの場合. AMDでは64
- SM内ではL1キャッシュ/シェアードメモリを共有(H100では256KB)
 - 配分は(ある程度)調節可能
 - シェアードメモリはCUDAでは使えるがOpenACCでは(明示的には)使えない機能
 - グローバルメモリよりも圧倒的に速い

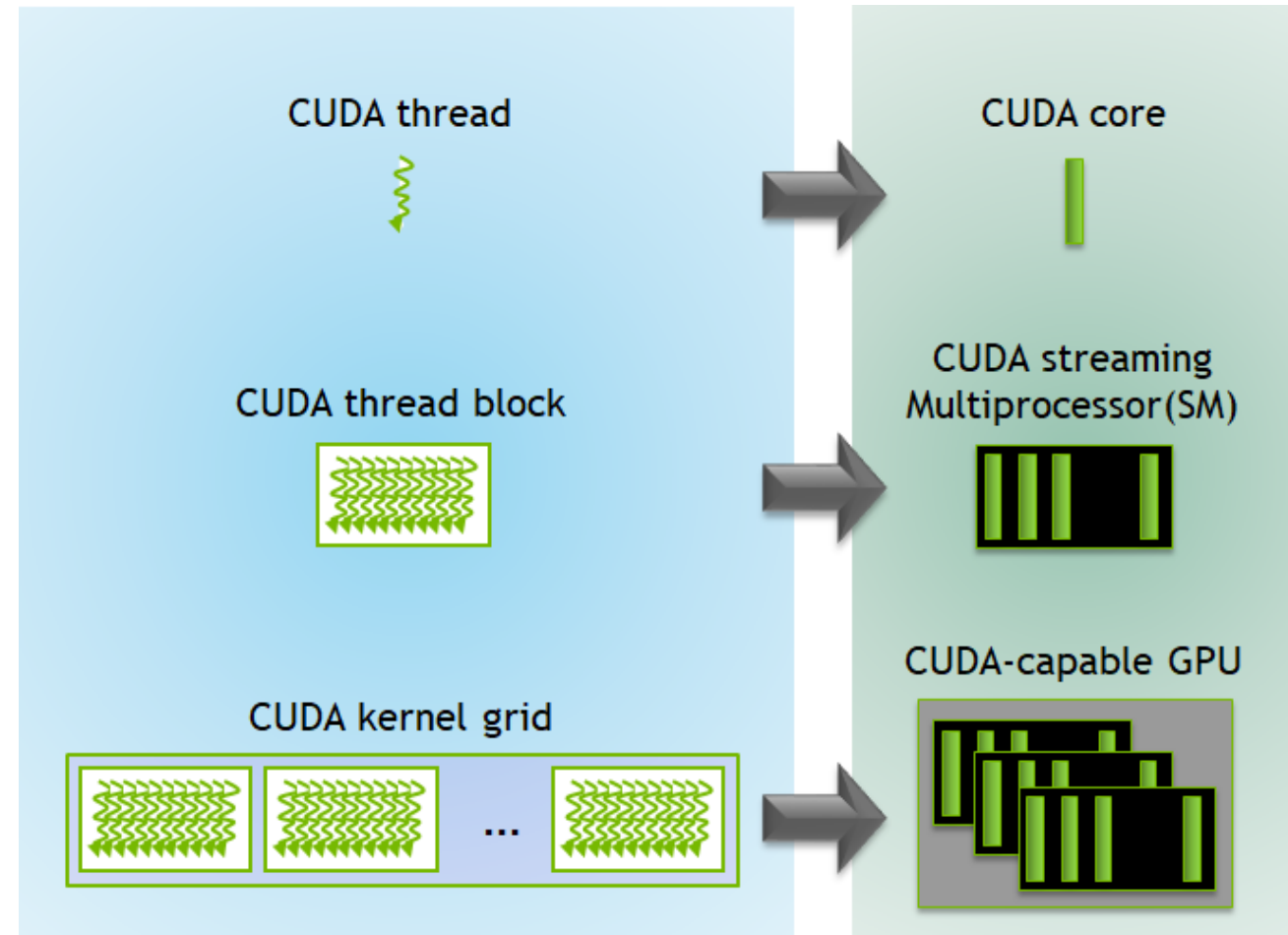


Section 2.

GPUプログラミングの基礎

GPUプログラミングでの基本思想

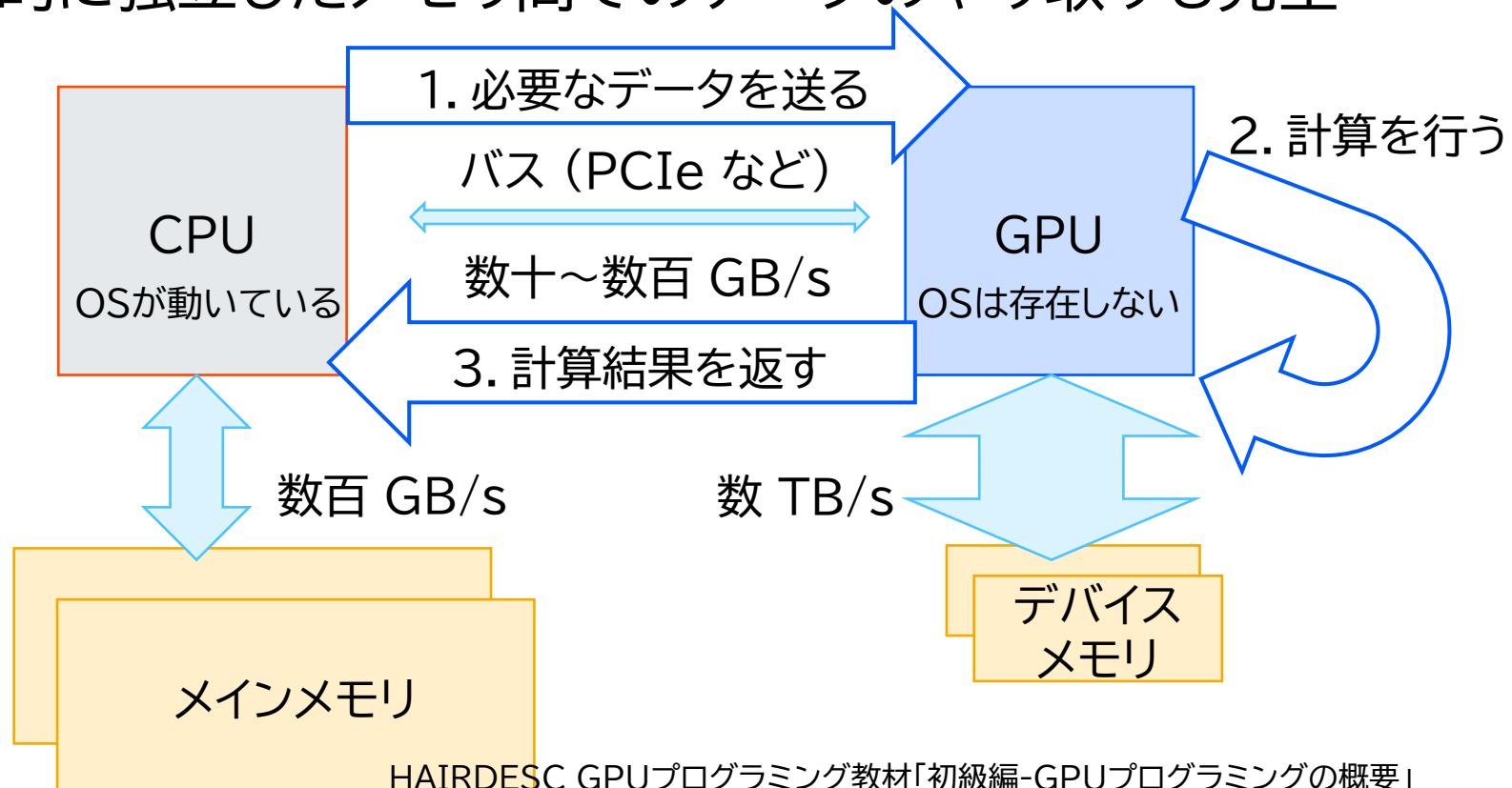
- ここではCUDA用語で説明
- 演算器の数よりも多数のスレッドを立てて計算
 - 各種レイテンシの隠蔽が目的
(このため、コア数の10倍以上の並列度があると性能を出しやすくなる)
 - スレッド間の同期は保証されない
(処理可能なものから随時処理する)
 - 各スレッドに分散された処理の実行順序も保証されない
 - スレッド間の同期処理が必要なら:
 - 演算カーネルを閉じる(指示文の場合)
 - 明示的な同期命令を実行(CUDAなど)
- スレッドブロックが基本実行単位
 - ブロックあたりのスレッド数が性能最適化上のキーパラメータ



<https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model/>

CPUとGPUのメモリ空間

- CPUとGPUは, 物理的に独立したメモリを持っていることがほとんど
 - (AMD MI300Aのような, CPUとGPUが物理的にメモリを共有する製品もある)
- (OSが動作している)CPUがGPU上の演算処理を起動
 - 物理的に独立したメモリ間でのデータのやり取りも発生



GPU化で高速化しやすいアプリの特徴

- GPUに搭載されている演算器を動かし続けられるだけの並列度がある
 - コア数の10倍以上の並列度があると, 比較的容易にGPU性能を引き出せる
- GPU側にデータを置きっぱなしにできる
 - CPU-GPU間のデータ転送がこまめに発生すると性能低下要因になる
 - AMD MI300Aでは, CPUとGPUが同じメモリを共有するので, この問題は起きない
 - NVIDIA GH200/GB200では, CPU-GPU間のデータ転送が高速なので問題になりにくい
- 各スレッドの演算処理にデータ依存性がない
 - 演算途中の結果を用いたif文などで処理が分裂すると, GPUで性能を出しづらい
 - 数十スレッド単位では共通の処理にまとめられるのであれば問題ない

GPU化で高速化しやすいアプリ例① (C)

- ・拡散方程式コード(C言語)の抜粋
 - ・メモリ律速な問題の代表例

```
int main(int argc, char *argv[])
{
    const int nx = 512;
    const int ny = 512;
    const int nz = 512;

    float *f = (float *)malloc(sizeof(float) * 1n);
    float *fn = (float *)malloc(sizeof(float) * 1n);

    for (; (icnt < nt) && (time + 0.5 * dt < 0.1); icnt++) {
        diffusion3d(nx, ny, nz, mgn, dx, dy, dz, dt, kappa, f, fn);

        swap(&f, &fn);
        time += dt;
    }

    return 0;
}
```

GPUのコア数よりも
十分に大きな問題サイズ

演算をGPUに閉じ込められる
ので、こまめなCPU-GPU間
データ転送は不要

```
void diffusion3d(int nx, int ny, int nz, int mgn, float dx, float
dy, float dz, float dt, float kappa, const float *f, float *fn)
{
    #pragma omp parallel for
    for(int k = 0; k < nz; k++) {
        for (int j = 0; j < ny; j++) {
            for (int i = 0; i < nx; i++) {
                const int ix = nx * ny * (k + mgn) + nx * j + i;
                const int ip = i == nx - 1 ? ix : ix + 1;
                const int im = i == 0 ? ix : ix - 1;
                const int jp = j == ny - 1 ? ix : ix + nx;
                const int jm = j == 0 ? ix : ix - nx;
                const int kp = (k == nz - 1) ? ix : ix + nx*ny;
                const int km = (k == 0) ? ix : ix - nx*ny;

                fn[ix] = cc*f[ix] + ce*f[ip] + cw*f[im] +
                cn*f[jp] + cs*f[jm] + ct*f[kp] + cb*f[km];
            }
        }
    }
}
```

各スレッドが
共通の処理を実行,
依存関係もない

GPU化で高速化しやすいアプリ例② (C++)

- N体計算コード(C++)の抜粋
 - 演算律速な問題の代表例

```
auto main(void) -> int32_t {
    const auto num = 1048576;

    float4 *pos = nullptr;
    float3 *vel = nullptr;
    float4 *acc = nullptr;
    allocate_Nbody_particles(&pos, &vel, &acc, num);

    while (present < snp_fin) {
        // orbit integration
        drift(num, pos, vel, dt);
        calc_acc(num, pos, acc, num, pos, eps2);
        trim_acc(num, acc, pos, eps_inv);
        kick(num, vel, acc, dt);

        // write snapshot
        if (present > previous) {
            write_snapshot(num, pos, vel, acc, file, present, time);
        }
    }

    return (0);
}
```

GPUのコア数よりも十分に大きな問題サイズ

演算をGPUに閉じ込められるので、こまめなCPU-GPU間データ転送は不要

```
static inline void calc_acc(const int32_t Ni, const float4
*const ipos, float4 *__restrict iacc, const int32_t Nj,
const float4 *const jpos, const float eps2) {
#pragma omp parallel for
    for (int32_t i = 0; i < Ni; i++) {
        const auto pi = ipos[i];
        float4 ai = {0.0F, 0.0F, 0.0F, 0.0F};

        for (int32_t j = 0; j < Nj; j++) {
            const auto pj = jpos[j];

            // particle-particle interaction
            const auto dx = pj.x - pi.x;
            const auto dy = pj.y - pi.y;
            const auto dz = pj.z - pi.z;
            const auto r2 = eps2 + dx * dx + dy * dy + dz * dz;
            const auto r_inv = 1.0F / std::sqrt(r2);
            const auto r2_inv = r_inv * r_inv;
            const auto alp = pj.w * r_inv * r2_inv;

            // accumulation
            ai.x += alp * dx;
            ai.y += alp * dy;
            ai.z += alp * dz;
            ai.w += alp * r2;
        }
        iacc[i] = ai;
    }
}
```

各スレッドが共通の処理を実行、依存関係もない

GPU化で高速化しやすいアプリ例① (F)

- ・ 拡散方程式コード (Fortran) の抜粋
 - ・ メモリ律速な問題の代表例

```

program main
  use diffusion
  implicit none

  integer,parameter :: nx0 = 512
  integer,parameter :: ny0 = 512
  integer,parameter :: nz0 = 512
  real(KIND=4),pointer,dimension(:,:,:) :: f,fn

  allocate(f(nx,ny,0:nz+1))
  allocate(fn(nx,ny,0:nz+1))

  do icnt = 0, nt-1
    diffusion3d(nx, ny, nz, dx, dy, dz, dt, kappa, f, fn)

    call swap(f, fn)
    time = time + dt
  end do

  deallocate(f,fn)
end program main
    
```

GPUのコア数よりも
十分に大きな問題サイズ

演算をGPUに閉じ込められるので、こまめなCPU-GPU間データ転送は不要

```

module diffusion
  implicit none
  contains
  double precision function diffusion3d(nx, ny, nz, dx, dy, dz, dt, kappa,
    f, fn)
    !$omp parallel do private(i,j,k,w,e,n,s,b,t)
    do k = 1, nz
      do j = 1, ny
        do i = 1, nx
          w = -1; e = 1; n = -1; s = 1; b = -1; t = 1;
          if(i == 1) w = 0
          if(i == nx) e = 0
          if(j == 1) n = 0
          if(j == ny) s = 0
          if(k == 1) b = 0
          if(k == nz) t = 0
          fn(i,j,k) = cc * f(i,j,k) + cw * f(i+w,j,k) &
            + ce * f(i+e,j,k) + cs * f(i,j+s,k) + cn * f(i,j+n,k) &
            + cb * f(i,j,k+b) + ct * f(i,j,k+t)
        end do
      end do
    end do
  end function diffusion3d
end module diffusion
    
```

ループ内にif文があってもOK
(この例では処理内容が共通)

各スレッドが
共通の処理を実行、
依存関係もない

Section 3.

GPUプログラミング手法

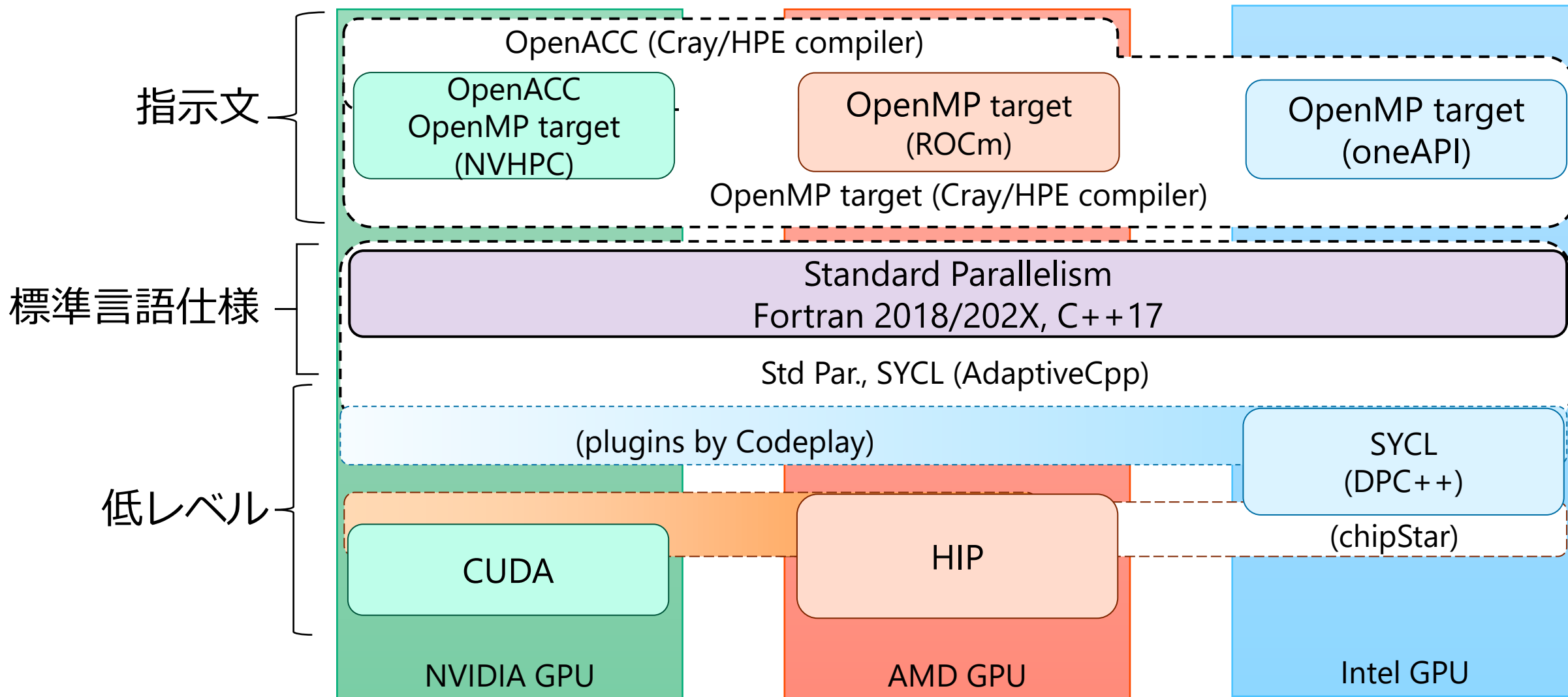
GPUスパコンの近況

- GPUは最近のスパコンでのメジャーな演算加速器(特に上位システムで顕著)
- 「GPU = NVIDIAのGPU」と言って良いぐらいの独占状態
 - 国内ではHPCIに資源提供されているほとんどのGPUスパコンはNVIDIA製
 - 唯一の例外は筑波大CCSのSirius(AMD MI300A搭載)
 - 海外のハイエンドスパコンではAMD, Intel製GPUも採用
 - Frontier などAMD製GPUを搭載したシステム, AuroraなどIntel製GPUを搭載したシステム
 - TOP500の上位にAMD, Intel, NVIDIA製GPUがランクイン
- GPUベンダー間の競争が活性化
 - 発散するプログラミング環境への対応も必要
- 今後国内で導入されるスパコンはどんなもの？
 - GPU搭載スパコンの採用事例は今後も増えていく傾向であり, GPU移植は必須と言える
 - ポスト富岳(富岳NEXT)は演算加速器(NVIDIA製GPU)を搭載
 - HPCI構成機関が今後どのようなシステムを入れてくるかは不明, 拠点ごとの多様性も？
 - 求める性能, 払えるプログラミングコスト, 対象とするGPUベンダーを勘案して採用するGPUプログラミング手法を選定する必要がある

GPU向けのプログラミング環境

2024年2月のPCCC AI/HPC OSS活用WSでの講演資料

(https://www.pccluster.org/ja/event/data/240205_pccc_wsAI-HPC-OSS_06_hanawa-miki.pdf)



GPUプログラミング手法の比較表

- Fortran の「OK」は動作するという意味. 性能については別問題

手法	ベース言語など	強み	弱み	Fortran対応	対象GPUベンダー
CUDA C++	C++	詳細な最適化可能 最新機能が使える	記述量が多い GPU専用コード	CUDA Fortran	NVIDIA限定
HIP	C++	詳細な最適化可能 ほぼCUDA C++	記述量が多い GPU専用コード	hipfort	AMD, NVIDIA (Intel: chipStar?)
SYCL	C++	詳細な最適化可能 ラムダ式	記述量が多い ラムダ式	N/A	Intel, NVIDIA, AMD
OpenACC	指示文	移植コスト低 CPUコードと共通化可能	CUDAより遅い (メモリ律速なら それなり?)	OK	ほぼNVIDIA限定 (HPE Cray コンパイラ はAMDも対応)
OpenMP (target指示文)	指示文	移植コスト低 CPUコードと共通化可能	CUDAより遅い (メモリ律速なら それなり?)	OK	NVIDIA, AMD, Intel
言語の標準規格	C++17以降 Fortran 2008	CPUでも同じ コードが動く	多くの制約あり CUDAより遅い	Fortran 2008	NVIDIA, AMD, Intel

大まかな考え方の違い

- 簡易GPU化(OpenACC, OpenMP target, C++17, ...)
 - 並列化可能なループであることをコンパイラに伝える
(自明な並列度があるループであるので, 簡単にGPU化できるループ)
 - 「どうGPU化するか」はコンパイラ任せ
 - なるべくコンパイラの自由度を増やしてあげる方が速くなる傾向
 - 初心者・初級者向け. まずはこちらを推奨
- 自力でのGPU化(CUDA, HIP, SYCL, ...)
 - 「どうGPU化するか」を自分で考え, 自分で実装
 - どのような演算命令を使うか, など自分で何でもできる
 - OpenACC, OpenMP target, C++17ではGPU化が難しいループであってもGPU化できる
 - 例: 重力ツリーコードの完全GPU実装は指示文では手に負えないが, CUDAなどでは実装可能
 - 中・上級者向け. 指示文でできなかった時にこちらの採用を検討

指示文ベースでGPU化したい場合の選択肢

• OpenACC

- GPU向けのメジャーな指示文
- PGIがNVIDIAに買収された結果, NVIDIA色が強くなった
- AMD, Intelはサポートしていない→直接支援が受けられない
 - HPE Crayコンパイラであれば, AMD GPU向けのOpenACCもサポート
(国内のAMD GPU搭載スパコンのベンダーはNECのため, このオプションは取れない)
 - IntelはOpenACCからOpenMP targetへの変換ツールを開発

• OpenMPのtarget指示文

- OpenMP 4.0以降でアクセラレータへのオフロードがサポート
- OpenMP 5.0で loop 指示節が追加, OpenACCの実装も可能に
- NVIDIA, AMD, Intel 全てのGPU向けにサポートされる
- 現時点ではOpenACCの全ての機能に対応できていない
 - 非同期実行の(細やかな)制御など

ベンダーニュートラル実装

- 汎用CPU向けに開発されてきたプログラムは、異なるベンダー製CPUや異なるアーキテクチャ(x84/aarch64)のCPUでも動作した
- GPUプログラムでは、用いたプログラミング手法によっては特定ベンダー製GPUの上でしか動作しない = ベンダーロックイン
 - CUDA: NVIDIA製GPU専用の開発環境
 - OpenACC: 実質的にNVIDIA製GPU向けの指示文
- ベンダーニュートラル実装できると何が嬉しいか？
 - 自分がアクセスできるシステムが増える
 - (特にOSSでは)興味を持った人がアクセスできる・動かしたい環境と開発者の環境が一致しない場合もあるので、リーチできるユーザー層が広がる
- ベンダーニュートラル実装可能なプログラミング環境例
 - 指示文ベース: OpenMP target
 - C++ラムダベース: Kokkos, RAJA, SYCL

Section 4.

関連情報

関連するHAIRDESC資料

- 指示文ベースで単体GPU化に着手する方向け
 - (OpenMPを用いたスレッド並列化済みコードからのGPU移植を想定)
 - 『初級編 単一GPUプログラミング』
 - OpenACC, OpenMP targetそれぞれが別資料となっている
- コードのGPU化・性能最適化に取り組む全ての方向け
 - GPUの性能を十分に引き出せているかを定量的に評価するためには測定が必須
 - GPU化したコードがそもそもGPU上で動作しているか, も確認できる
 - 『性能チューニング編 プロファイリング』
 - (マルチGPUコードのプロファイリングについては今後資料提供予定)
- 指示文ベースでGPU化されたコードのマルチGPU化に着手する方向け
 - 『中級編 マルチGPUプログラミング』
 - MPIを用いたマルチGPU化方法を解説
 - GPU化についてはOpenACC, OpenMP target のどちらか(別資料)を使用

資料提供者一覧

- 五十音順, 敬称略
- 下川辺 隆史(東京大学 情報基盤センター)
 - Section 2
- 塙 敏博(東京大学 情報基盤センター)
 - Sections 1, 3
- 星野 哲也(名古屋大学 情報基盤センター)
 - Section 2
- 三木 洋平(東京大学 情報基盤センター)
 - Sections 1, 2, 3
- 山崎 一哉(東京大学 情報基盤センター)
 - Section 2

資料作成者一覧

- 五十音順, 敬称略
- 三木 洋平(東京大学 情報基盤センター)
 - 2026/3/25版